Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST
Training Images: 60,000
Testing Images: 10,000
Classes: 10
Image Size: 28×28

4. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

5. Source Code

GitHub Repo Link: `https://github.com/Hasini-Kotha/Deep_Learning`

Kaggle Notebook Link: `https://www.kaggle.com/code/kothahasini/deep-learning-exp2` **Google Colab Notebook Link(for XOR implementation using MLP):** `https://colab.research.google.com/drive/125kJQkQ0NYj8i82VncySdVSTGVK4Usrv?usp=sharing`

6. Experimental Procedure

Task 1: Dataset Exploration
- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing
- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction
Construct an MLP with
- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training
Compile using
- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation
Compute
- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

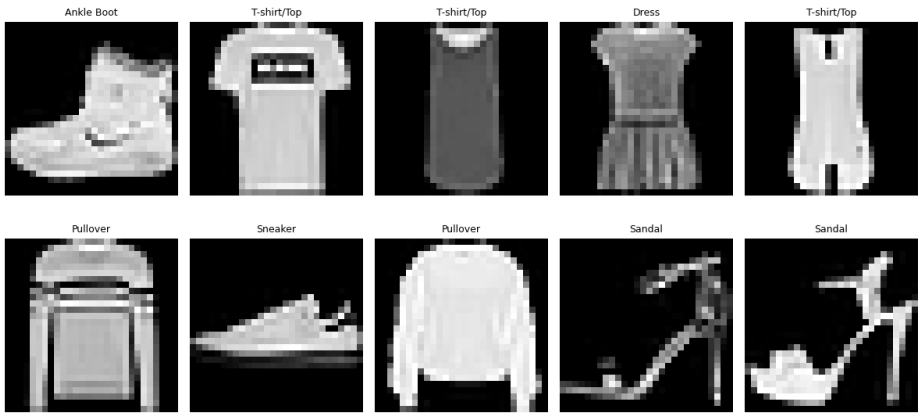
Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

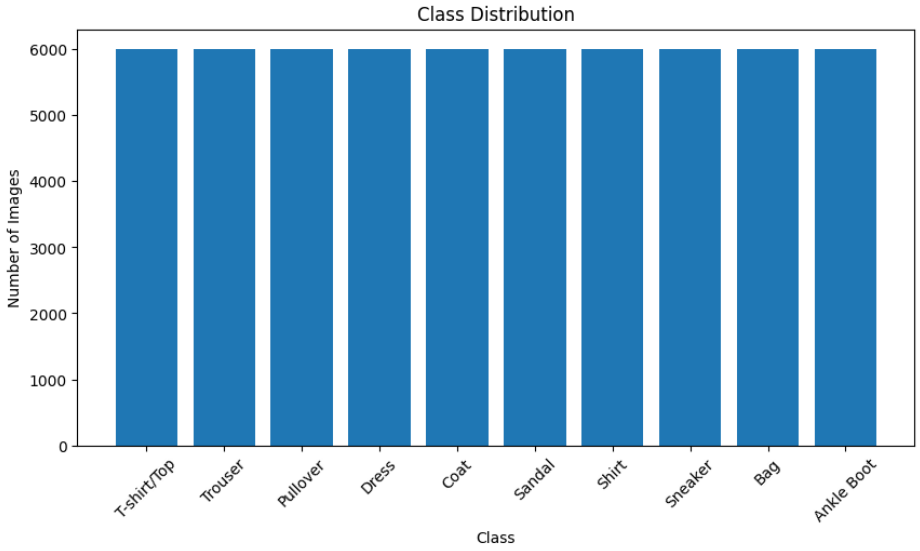
8. Mandatory Plots

8.1 Sample Images



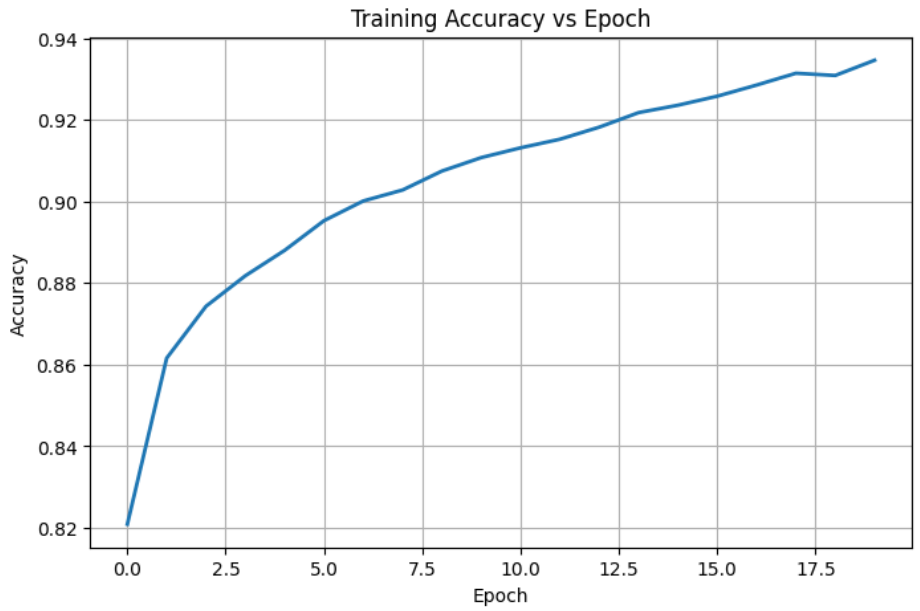
Inference: This plot displays a random selection of grayscale images from the Fashion-MNIST dataset,it verifies that the data is loaded correctly and the items are easily recognizable and have diverse categories like trousers ,pullovers and ankle boots.

8.2 Class Distribution



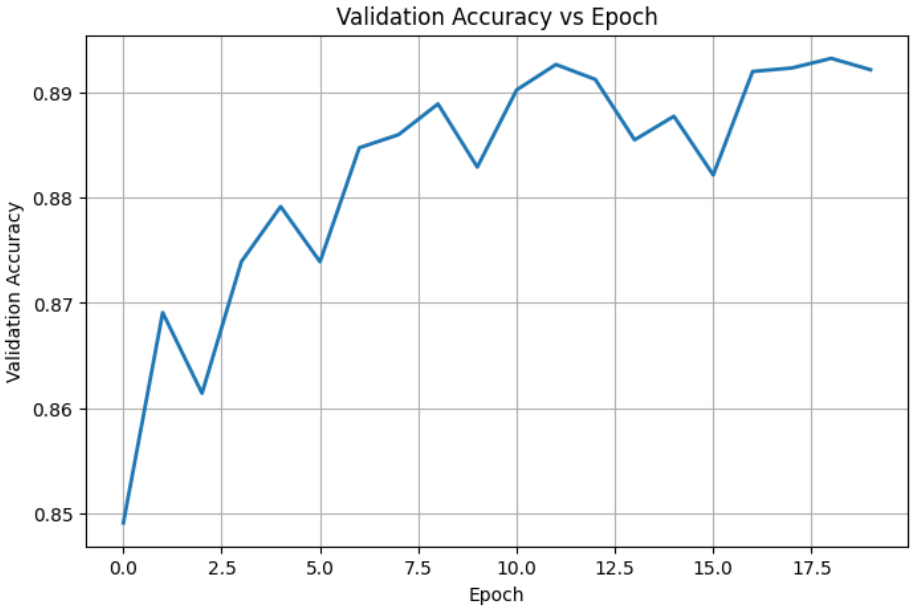
Inference: The bar chart reveals that the dataset is perfectly balanced across all 10 categories, with exactly 6,000 samples for each class in the training set.This balance ensures that the model will not become biased towards any majority of the class during the traning.

8.3 Training Accuracy vs Epoch



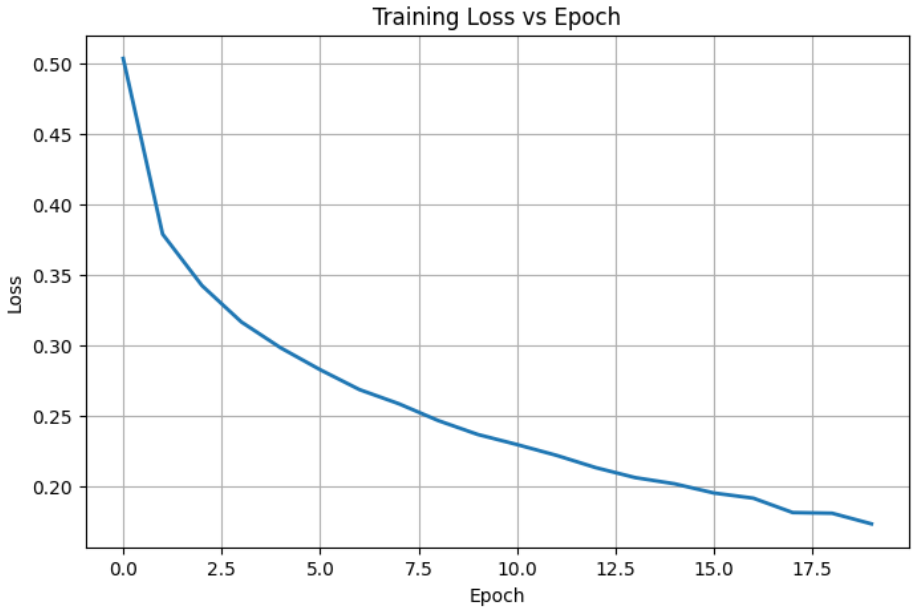
Inference: The training accuracy steadily climbs from roughly 82% in the first epoch to over 93% by epoch 20, This consistent upward trend says that the models capacity is suffient to effectively learn the patterns within the training data without underfitting.

8.4 Validation Accuracy vs Epoch



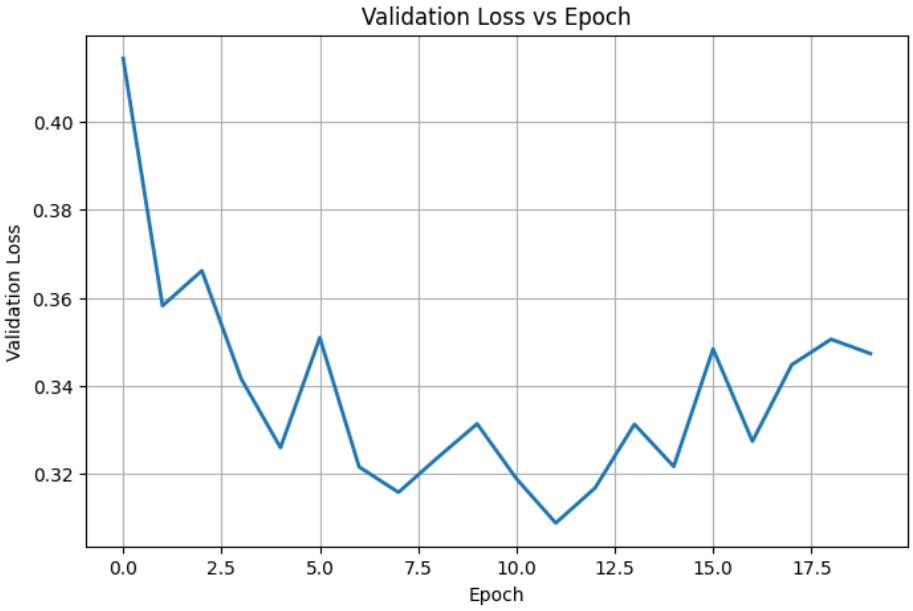
Inference: The validation accuracy experiences a sharp initial increase before reducing around 89%. The growing gap between the training accuracy (93%) and this validation accuracy curve suggests that the model begins to slight overfit the training data in the later epochs.

8.5 Training Loss vs Epoch



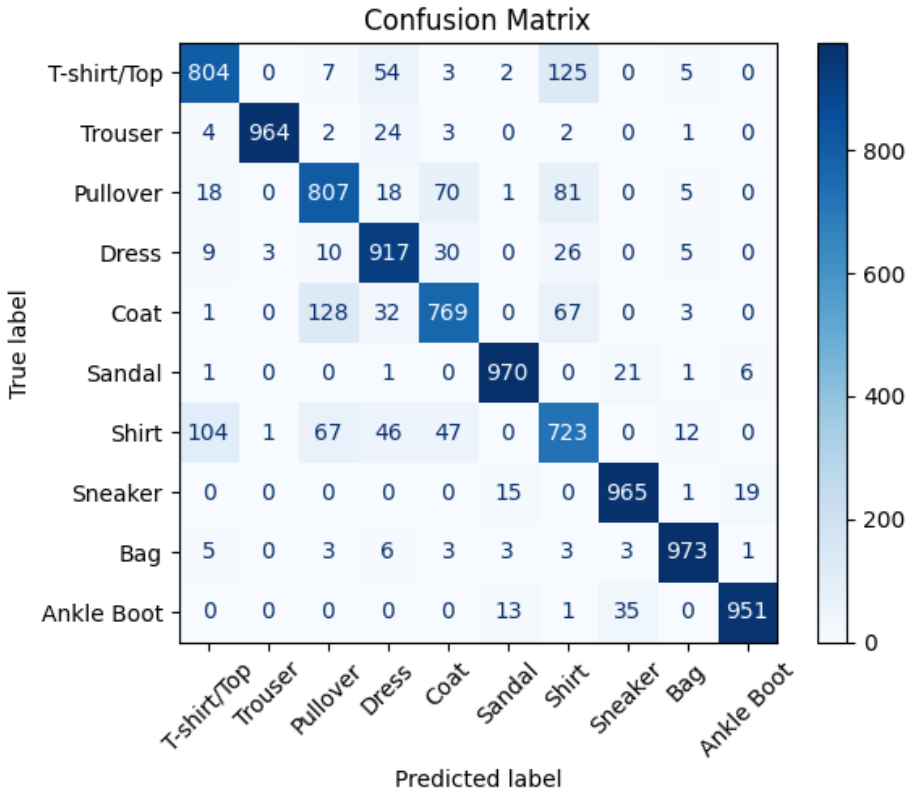
Inference: The training loss curve shows a smooth and continuous exponential decay throughout the 20 epochs. This behavior indicates that the Adam optimizer is functioning correctly and consistently minimizing the categorical cross-entropy loss on the training set.

8.6 Validation Loss vs Epoch



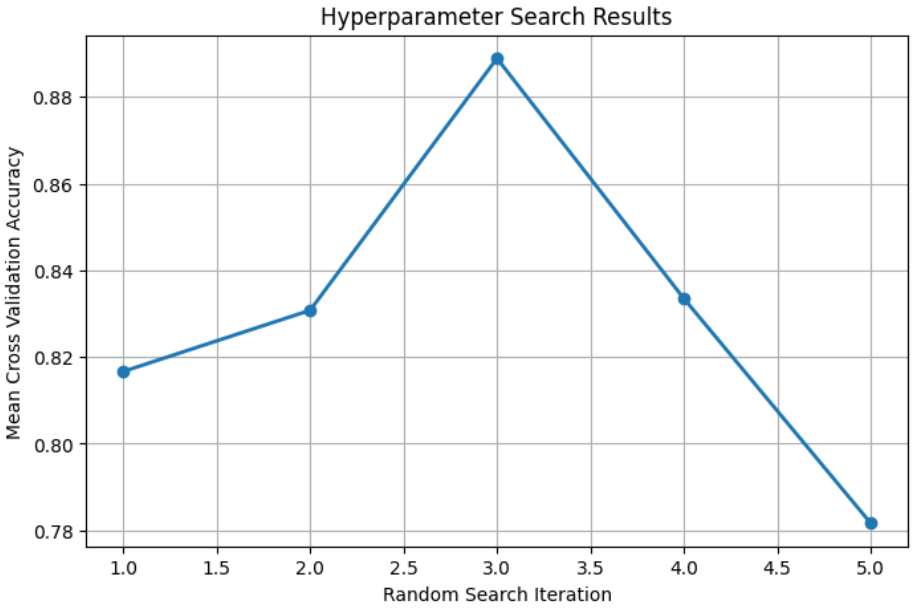
Inference: Unlike the training loss, the validation loss drops initially but then exhibits fluctuations and a slight increase after the 5th epoch. This divergence from the training loss is a classic symptom of overfitting, where the model starts memorizing noise rather than generalizing so it slightly got higher after the 5th epochh.

8.7 Confusion Matrix



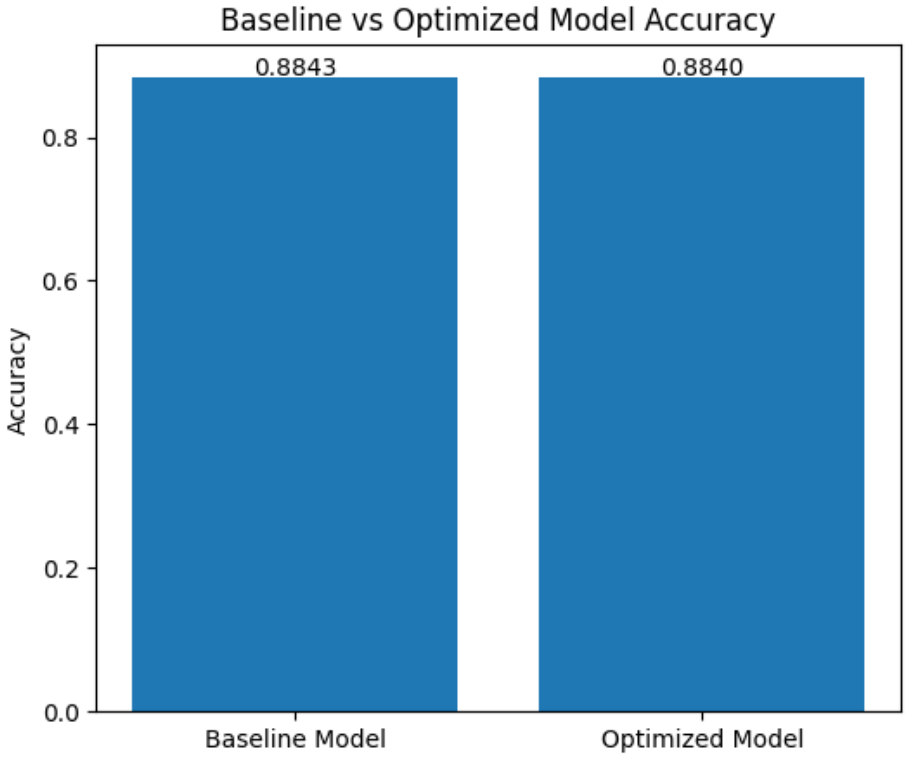
Inference: The strong dark diagonal indicates that the model classifies the vast majority of items. However, the lighter off-diagonal squares reveal that it somtimes struggles to differntiate visually similar upper-body garments, frequently confusing "Shirt" with "T-shirt/Top.

8.8 Hyperparameter Search Results



Inference: This plot tracks the mean cross-validaton accuracy across 5 different randomized search iteration,It highlights the variance in model performance based on different hyperparameter configurations, ultimately identifying an optimal setup that achieved nearly 89% cross-validation accuracy.

8.9 Best Model Accuracy Comparison



Inference: The comparison shows that the hyperparameter optimization yielded a model with performance nearly similar to baseline model (both around 88.4% testing accuracy). This suggests that the initial, intuitive baseline configuration is already highly effective for this specific classification task.

9. Results

Best Hyperparameters

Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	RMSprop
Activation Function	Tanh
Epochs	30
Dropout	0.2
Cross-validation Accuracy	88.89%
Testing Accuracy	88.40%

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8843	0.8840
Precision	0.8853	0.8842
Recall	0.8843	0.8840
F1-score	0.8843	0.8838
Training Time	∼1.5 minutes	∼20 minutes

10. Discussion

1. **Which optimization method was used?**
   Randomized Search using **RandomizedSearchCV** together with the **SciKeras** wrapper was used fpr hyperparameter optimisation ,instead of evaluating every hyperparamter combination , randomised search samples a subset of combinations making it computationally efficent while still identifying a slightly high performing model.
2. **Which hyperparameters were selected?**
   The best hyperparameter combination obtained from the search was:
   - Hidden Layers: 3
   - Hidden Neurons: 128
   - Learning Rate: 0.001
   - Batch Size: 32
   - Optimizer: RMSProp
   - Activation Function: Tanh
   - Epochs: 30
   - Dropout Rate: 0.2
3. **Did optimization improve performance?**
   No
4. **Which hyperparameter had the greatest impact?**
   The optimizer, activation function, network depth, and number of training epochs had the greatest influence on performance. The optimized model selected RMSProp, Tanh activation, three hidden layers, and 30 epochs, suggesting that these settings provided better feature learning for the Fashion-MNIST dataset.
5. **Compare Grid Search and Randomized Search.**
   Grid Search evaluates every possible combination of hyperparameters within the search space, making the grid search computationally expensive for large search spaces ,on other side Randomized search evaluates only a randmly selected subset of combinations ,which significantly reduces computational cost and time while stil producing competitive results so randomised search is more suitable for large-scale hyperparamter optimisation problems.
6. **Recommend the best MLP configuration.**
   Based on the experimental results, the recommended MLP configuration consists of three hidden layers with 128 neurons each, the Tanh activation function, the RMSProp optimizer with a learning rate of 0.001, a batch size of 32, 30 training epochs, and a dropout rate of 0.2.this configuration achived the highest accuracy among the evaluted models while stil maintaing good overall classification performance.

11. XOR Implementation using MLP

To demonstrate that a Multi-Layer Perceptron (MLP) can solve problems that are not linearly separable, an MLP was trained on the XOR dataset for 50 epochs. The network architecture consisted of one

hidden layer with 2 neurons (using a suitable activation function like Sigmoid) and an output layer with 1 neuron. The decision boundaries below illustrate the learning process.

Decision Boundaries

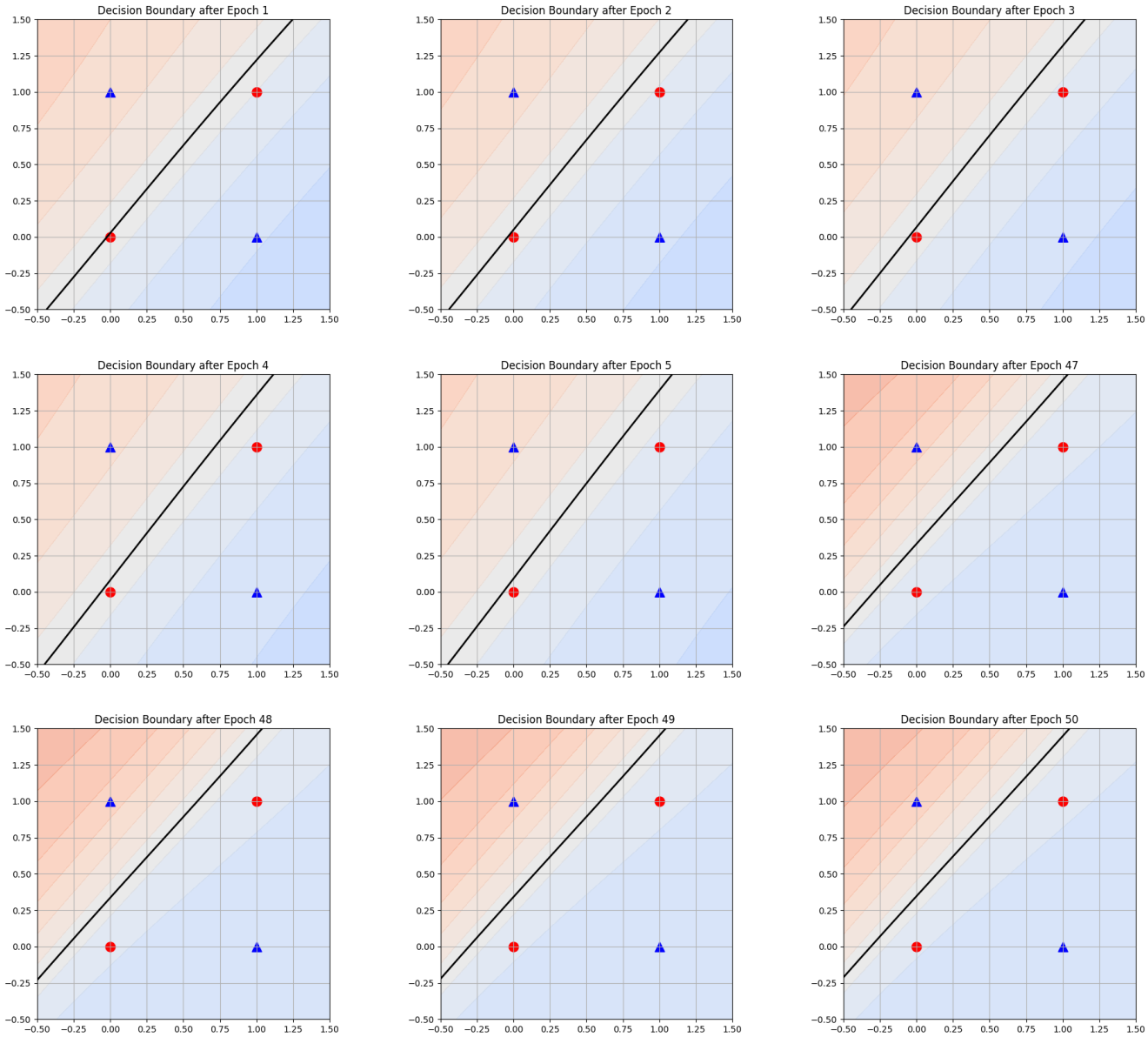


Figure 1: Evolution of the MLP Decision Boundary for the XOR Gate. The first five plots show the initial epochs (1-5), and the last four show the final epochs (47-50).

Final Predictions (Epoch 50)

Input	Target	Predicted Probability	Output Class
$(0, 0)$	0	0.4327	0
$(0, 1)$	1	0.6649	1
$(1, 0)$	1	0.3986	0
$(1, 1)$	0	0.4355	0

Final Accuracy: 0.75

Observation: Unlike a Single Layer Perceptron, the Multi-Layer Perceptron introduces a hidden layer that allows it to learn non-linear decision boundaries. The provided predictions show that after 50 epochs,the MLP correctly classifies 3 out of 4 samples (achiveing 75% accuracy), successfully shifting its decision boundary to group the XOR inputs.

12. Conclusion

In this experiment, a Multi-Layer Perceptron (MLP) was successfully implemented and evaluated on the Fashion-MNIST dataset. The initial baseline model performed effectively with a testing accuracy of 88.43%. We then performed automated hyperparameter optimization using Randomized Search, exploring various network depths, activation functions, optimizers, and learning rates. The optimized configuration (with 3 hidden layers with 128 neurons each, Tanh activation, RMSprop optimizer, learning rate 0.001, batch size 32, 30 epochs, and 0.2 dropout) achieved a testing accuracy of 88.40%, along with robust precision, recall, and F1-scores which confirms that the model architectures is highly effective for image classification and it demonstrated the utility of automated hyperparameter tuning , for identifying optimal training configurations. Furthermore, the capacity of an MLP to learn non-linear boundaries was observed by training it on the XOR dataset.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.